

File Upload Vulnerabilities — Interview Prep Notes

Unrestricted File Upload

1. Logical description

- App accepts user-supplied file (name, extension, content, MIME, size) without adequate validation of type/content/destination/execution-context.
- Root issue: trust boundary violation — attacker controls filename/extension/content-type/bytes, server stores and/or later executes it.
- Taint model: source = upload form/API → sink = filesystem write (+ optional execution) → missing sanitizer = extension/MIME/magic-byte/AV/exec-context check.

2. OWASP category & CWE

- OWASP 2021: **A04:2021-Insecure Design** (primary, missing threat-model control); also overlaps **A05:2021-Security Misconfiguration** (permissive server exec config) — debated mapping, call out both in interview.
- CWE-434 (Unrestricted Upload of File with Dangerous Type) — primary
- CWE-646 (Reliance on File Name/Extension of Externally-Supplied File)
- CWE-73 (External Control of File Name or Path) — when destination path attacker-influenced
- CWE-22 (Path Traversal) — when filename allows `../` to write outside intended dir
- CWE-98 — if uploaded file later included/executed via LFI

3. Common variants

- Extension bypass (double ext `shell.php.jpg`, null byte `shell.php%00.jpg`, case `shell.Php`, trailing dot/space `shell.php.`, alt exec ext `.phtml/.php5/.pht/.asp;.jpg`)
- MIME/Content-Type spoofing (client sets `image/png` header, body is PHP)
- Magic byte bypass (valid image header + PHP payload appended = polyglot)
- Path traversal in filename → overwrite arbitrary file (`../../../../htaccess`)
- Race condition (upload → scan/delete window; access before deletion — TOCTOU)
- Zip/archive upload → Zip Slip (path traversal on extract) → RCE
- Overwrite of protective config (`.htaccess`, `web.config`) to enable execution
- SVG upload → stored XSS / XXE (SVG is XML)
- ImageMagick/GD processing → ImageTragick-style RCE via crafted image metadata
- Oversized/zip-bomb upload → DoS
- Client-side-only validation bypass (JS check, no server check)
- Upload to publicly accessible, executable-by-webserver directory

4. Attack surface / entry points

- ☐ Profile picture / avatar upload
- ☐ Document/resume/KYC upload

- ☐ Import features (CSV/XML/JSON bulk import)
- ☐ Chat/attachment upload
- ☐ Rich text editor image embed
- ☐ API multipart/form-data endpoints
- ☐ Admin panel theme/plugin/logo upload
- ☐ Bulk/zip upload with server-side extraction
- ☐ WebDAV / PUT-enabled endpoints
- ☐ Third-party plugin upload handlers (CMS/LMS)

5. Technical testing workflow

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/why
Map upload points	Where uploads exist, tech stack, storage path	Crawl, view-source, JS bundle grep	Site map + Target scope	Spider/AJAX Spider	<code>file-upload</code> tag templates	N/A
Extension filter	Server checks ext or only client	Upload <code>.php</code> , <code>.phtml</code> , <code>.php5</code> , <code>.pht</code> , <code>.asp</code> , <code>.jpg</code>	Repeater — swap ext	Manual Request Editor	Custom template with ext fuzz list	Alt-exec extensions bypass blacklist
Content-Type/MIME check	Server trusts header vs content	Change <code>Content-Type</code> to <code>image/jpeg</code> , keep PHP body	Repeater edit multipart headers	Manual Request Editor	—	MIME spoof tests server logic depth
Magic byte/polyglot	Content sniffing depth	Prepend <code>GIF89a;</code> / valid PNG header to PHP payload	Repeater, hex editor	Manual editor	—	Polyglot bypasses signature-based filters
Null byte / double ext	Legacy parsing bugs	<code>shell.php%00.jpg</code> , <code>shell.php.jpg</code>	Repeater	Manual editor	—	Exploits truncation bugs in older stacks
Path traversal in filename	Destination path control	<code>../../../../shell.php</code> as filename in multipart	Repeater	Manual editor	LFI/traversal templates	Write outside intended upload dir
Storage location & executability	Is upload dir web-accessible + interpretable	Request uploaded file path directly, try execute	Repeater, Intruder for filename brute	Forced Browse	<code>exposed-payloads</code> / dir brute templates	Confirms exec-in-place
Race condition	Scan-then-delete window	Rapid parallel upload+request via script	Turbo Intruder	ZAP scripting	—	Exploit TOCTOU before AV scan deletes file
Zip Slip	Archive extraction traversal	Craft zip with <code>../../../../</code> entry names (Python <code>zipfile</code>)	Repeater upload, manual extract check	Manual editor	—	RCE via extraction path traversal
SVG/XXE	XML-based file abused	Upload SVG with XXE payload, <code><script></code>	Repeater	Manual editor	XXE templates	Stored XSS/XXE via image pipeline
Config overwrite	Can overwrite <code>.htaccess</code> / <code>web.config</code>	Upload file named <code>.htaccess</code> with exec directive	Repeater	Manual editor	—	Enable execution of restricted ext

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/why
Confirm RCE	Full attacker loop	Access uploaded webshell URL, run <code>id / whoami</code> <code>i</code>	Repeater/Browser	Browser	—	Impact confirmation, not just "upload succeeded"

6. Evidence to collect

- Full upload request/response (headers, multipart body, filename, Content-Type)
- Screenshot/response of accessible uploaded file URL
- Command output (`id`, `whoami`, `hostname`) from webshell execution
- Server response headers confirming interpreter executed file (not served as static text)
- Storage path disclosed (via error or response)
- Video/PoC script for race condition or zip slip

7. Good PoC

- Step-by-step repro: request used, filename/bypass technique, resulting accessible URL
- Screenshot of webshell executing `id / whoami / system('id')`
- Impact statement: RCE achieved, full server compromise possible
- curl one-liner for reproducibility, e.g.:

```
curl -F "file=@shell.phtml;type=image/jpeg" https://target/upload
curl https://target/uploads/shell.phtml?cmd=id
```

8. Attack chains

- Upload → RCE (webshell) → lateral movement / full compromise
- Upload SVG → Stored XSS → session hijack / account takeover
- Upload → Path traversal write → LFI read of written file → RCE (log/asset poisoning)
- Upload zip → Zip Slip → overwrite app code/config → RCE
- Upload → overwrite `.htaccess` / `web.config` → re-enable script execution → RCE
- Upload malicious archive → decompression DoS (zip bomb)
- Upload + IDOR on file retrieval → other users' sensitive uploaded docs exposed

9. Common false positives

- Upload "succeeds" (200 OK) but file stored with randomized/sanitized name — not executable
- File uploaded but stored outside webroot / non-executable mount
- WAF/AV silently strips payload, response looks like success
- Content-Disposition forces download instead of inline render (no XSS even if HTML uploaded)
- Extension accepted but MIME-sniffing at serve time still blocks execution (nosniff header, correct Content-Type on serve)
- Duplicate filename collision reported as "overwrite" but isn't attacker-controlled path

10. Prerequisites for exploitability

- Upload functionality accessible (with or without auth)

- Insufficient extension/MIME/content validation
- Uploaded file stored in web-accessible, interpretable directory
- Web server configured to execute that file type in that directory
- No AV/content-disposition/nosniff mitigations, or bypassable ones

11. Attacker achieves

- Remote Code Execution / full server compromise
- Stored XSS (via HTML/SVG upload)
- Defacement
- Malware/webshell hosting (reputational + supply-chain risk)
- Data exfiltration via webshell
- Pivot to internal network

12. CIA impact (tabular, per variant)

Variant	C	I	A	Example CVSS v3.1 vector
Webshell RCE	High	High	High	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (9.8)
Stored XSS via SVG/HTML	Low	Low	None	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N (5.4)
Zip Slip → overwrite config	High	High	High	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H (9.9)
Zip bomb / DoS	None	None	High	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H (7.5)
Unrestricted upload, no exec path (info only)	Low	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (5.3)

13. Typical CVSS

- RCE-capable: **Critical, 9.8–9.9**
- Auth-required RCE: **High, 7.2–8.8** (`PR:L` or `PR:H`)
- Non-executable upload (storage/DoS only): **Medium, 5–6.5**

14. Business impact (financial app context)

- RCE on server hosting cardholder/PII data → direct PCI-DSS Req 6 (secure SDLC) & Req 11 violation
- Regulatory breach notification (GDPR Art. 33, RBI/local banking CERT reporting) if data exfil confirmed
- Webshell persistence → long-term undetected fraud/data theft, brand/regulatory trust damage
- Potential mandatory PCI re-assessment / fines, customer churn
- If chained to internal pivot: risk to payment processing systems, not just web tier

15. Indicators/signals

- Response accepts unexpected extensions/MIME with 200
- Uploaded file directly retrievable at predictable/guessable path
- No Content-Disposition/nosniff headers on serve
- Upload dir listed in `robots.txt` or discoverable via directory brute force
- App uses client-side-only JS validation (no server round-trip check observed)
- Error messages leak storage path/filesystem structure

16. Root cause (code-level)

- Pseudocode (vulnerable):

```
file = request.files['upload']
save_path = UPLOAD_DIR + file.filename    # attacker-controlled name & no ext check
file.save(save_path)                     # dir is web-servable & interpretable
```

- Failures: trusting client filename verbatim, blacklist instead of whitelist extension check, no content/magic-byte validation, storing in executable webroot path, no re-encode/normalize on save.

17. Remediation / secure design

- Whitelist allowed extensions + validate against actual file content (magic bytes/MIME sniffing library, not header)
- Generate server-side random filename (UUID), strip original name and extension mapping via lookup table
- Store uploads outside webroot / non-executable storage (S3 with no exec, separate CDN domain)
- Disable script execution in upload directory (`php_admin_flag engine off` , `Options -ExecCGI` , `RemoveHandler`)
- Enforce `Content-Disposition: attachment` + `X-Content-Type-Options: nosniff` on serve
- Re-encode images (strip metadata/re-render) to neutralize embedded payloads
- Size limits + AV/malware scanning (ClamAV, cloud AV) before making file accessible
- Validate archive extraction paths (reject entries with `../` , canonicalize + prefix-check) to prevent Zip Slip
- Principle of least privilege on storage service account

```
# Python (Flask) - safe pattern
import uuid, os
from werkzeug.utils import secure_filename
ALLOWED = {'png', 'jpg', 'jpeg', 'pdf'}
ext = filename.rsplit('.', 1)[-1].lower()
if ext not in ALLOWED or not is_valid_magic_bytes(file.stream):
    abort(400)
safe_name = f"{uuid.uuid4()}.{ext}"
file.save(os.path.join(NON_WEBROOT_DIR, safe_name))
```

```
// Java
String ext = FilenameUtils.getExtension(originalName).toLowerCase();
if (!ALLOWED_EXT.contains(ext) || !isValidMagicBytes(fileBytes)) throw new BadRequestException();
String safeName = UUID.randomUUID() + "." + ext;
Files.write(Paths.get(NON_WEBROOT_DIR, safeName), fileBytes);
```

```
// Node.js
const ext = path.extname(file.originalname).toLowerCase();
if (!allowedExt.includes(ext) || !isValidMagicBytes(file.buffer)) return res.status(400).end();
const safeName = `${uuidv4()}${ext}`;
fs.writeFileSync(path.join(NON_WEBROOT_DIR, safeName), file.buffer);
```

```
// C#
var ext = Path.GetExtension(file.FileName).ToLower();
if (!allowedExt.Contains(ext) || !IsValidMagicBytes(file)) throw new BadRequestException();
var safeName = $"{Guid.NewGuid()}${ext}";
await File.WriteAllBytesAsync(Path.Combine(nonWebrootDir, safeName), fileBytes);
```

18. Compensating controls (if root cause unfixed)

- WAF rule blocking known dangerous extensions/multipart patterns
- Reverse proxy denies execution for upload path (`location /uploads { deny script exec }`)
- Segregate upload storage on isolated host/bucket with no execution capability
- Rate limit + auth-gate upload endpoints
- Continuous file-integrity monitoring on upload directories

19. If seen repeatedly — what it indicates

- No secure file-handling library/standard adopted org-wide (each team reinvents upload logic)
- Missing secure SDLC checklist item for "any user file upload" feature
- No centralized/shared upload-handling service — same as recurring IDOR pattern, argues for a shared wrapper library across products

20. Upstream improvements

- Shared internal "SafeUpload" library (whitelist+magic-byte+random-name+non-exec storage) as mandatory dependency
- Semgrep/CodeQL taint rule: file.save()/write() sink fed by request filename without sanitizer
- Framework-level protections: Spring `MultipartFile` + custom validator, Django `FileExtensionValidator` or + storage `MEDIA_ROOT` outside webroot
- Design checklist item: "Does this feature accept file upload? → mandatory security review + SafeUpload lib usage"
- CI/CD: block deploy if upload dir found under executable webroot path (IaC/config lint)

21. Sample triage response

Confirmed: Unrestricted file upload allows storage and execution of a PHP webshell at `/uploads/<file>`, resulting in RCE as `www-data` (evidenced by `id` command output). Rated Critical (CVSS 9.8, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`). Escalating to engineering for immediate hotfix (disable execution in upload dir) and full remediation per SafeUpload library. Requesting confirmation of blast radius (shared hosting/other tenants) before disclosure timeline is set.

22. Sample developer remediation note

Issue: `/api/upload` saves files using client-supplied filename directly into a web-executable directory with no extension/content validation, enabling RCE via webshell upload (CWE-434). Fix: (1) Validate extension against whitelist + verify magic bytes server-side. (2) Rename to random UUID on save. (3) Move storage to non-executable path (or S3 with public-read only, no exec). (4) Disable script handlers in upload directory via server config. (5) Add `Content-Disposition: attachment` + `nosniff` on serve. See SafeUpload lib (`common-upload-lib`) for reference implementation — required for all new upload features per design checklist.

23. Real-world CVE

- **CVE-2023-4220** — Chamilo LMS unauthenticated arbitrary file upload (`bigUpload.php`) → stored XSS + RCE via webshell upload into web-accessible `/main/inc/lib/javascript/bigupload/files/` directory; PoC: upload `rce.php` with `system("id")`, curl the file path, get shell output. Chained N-day family (CVE-2023-34960 SOAP RCE) actively exploited in the wild — good real-world talking point for "chained findings" and "in-the-wild exploitation" narrative.